Threshold Fully Homomorphic Encryption on Zoo Network: GPU-Accelerated Confidential Computation

Privacy-Preserving Smart Contracts via TFHE and CKKS

Antje Worring & Zach Kelling

Zoo Labs Foundation

Version 2026.04

2026

Abstract

We present Zoo Network's integration of threshold fully homomorphic encryption (TFHE and CKKS) into a Layer 2 blockchain, enabling arbitrary computation on encrypted on-chain state. Zoo deploys programmable bootstrapping (TFHE) for boolean and integer circuits and approximate arithmetic (CKKS) for privacy-preserving machine-learning inference directly within smart contracts. By mapping the Number Theoretic Transform butterfly network onto GPU thread blocks, we achieve a **40× speedup** over CPU-only bootstrapping, reducing single-gate bootstrap latency from 12 ms to 0.3 ms. Ciphertext operations are distributed across validators via an Encrypt-to-Shares (E2S) threshold decryption protocol with a **67-of-100** default quorum, ensuring that no individual validator ever possesses a complete decryption key. Key shares are protected in transit with ML-KEM-768 post-quantum key encapsulation and anchored on Lux K-Chain. Formal correctness of the CKKS rescaling pipeline, TFHE noise budget tracking, and GPU scaling bounds are verified in Lean 4 (proofs reside in `Crypto/FHE/CKKS.lean`, `Crypto/FHE/TFHE.lean`, and `GPU/FHEScaling.lean`). We benchmark confidential DeFi order matching, sealed-bid auctions, and encrypted neural-network inference, demonstrating practical throughput for production deployment on Zoo T-Chain.

Contents

1 Introduction

Public blockchains expose every transaction and every byte of contract state to all participants. This transparency, while valuable for auditability, precludes entire classes of applications: sealed-bid auctions, private order matching, confidential voting, and machine-learning inference over sensitive data.

Two cryptographic families address this gap:

1. **Zero-knowledge proofs** (ZKPs) allow a prover to convince a verifier of a statement's truth without revealing witness data. ZKPs prove facts about plaintext but do not enable *computation* on encrypted data.
2. **Fully homomorphic encryption** (FHE) permits arbitrary computation directly on ciphertexts. The result, once decrypted, equals the output of the same computation on plaintexts [1].

FHE is strictly more expressive than ZKPs for on-chain privacy: a smart contract can evaluate functions over encrypted inputs without any party—including validators—ever observing plaintext values. Zoo Network combines two complementary FHE schemes:

- **TFHE** (Torus FHE): bit-level and small-integer operations with fast programmable bootstrapping, ideal for branching logic, comparisons, and access-control circuits.
- **CKKS** (Cheon–Kim–Kim–Song): approximate fixed-point arithmetic over vectors of real numbers, ideal for neural-network inference, statistical aggregation, and financial modelling.

Zoo's approach differs from prior work in three ways: (1) GPU-accelerated bootstrapping that reduces latency by 40×, making FHE practical inside block times; (2) threshold decryption integrated with Zoo T-Chain validators so that decryption requires 67-of-100 consensus; and (3) formal verification in Lean 4 of noise budgets, rescaling correctness, and GPU parallelism bounds.

This paper is organized as follows. Sections 2–3 describe the two FHE schemes and their on-chain encoding. Section 4 details GPU acceleration. Section 5 presents the threshold decryption protocol. Section 6 explains integration with T-Chain. Section 7 covers formal verification. Section 8 surveys applications, Section 9 provides benchmarks, and Section 10 discusses related work.

2 TFHE on Zoo

2.1 LWE Encryption

TFHE operates over the Learning With Errors (LWE) problem. A plaintext bit $m \in \{0, 1\}$ is encrypted as:

$$\mathsf{ct} = (\mathbf{a}, b) \in \mathbb{Z}_q^n \times \mathbb{Z}_q, \quad b = \langle \mathbf{a}, \mathbf{s} \rangle + m \cdot \lfloor q/2 \rfloor + e \tag{1}$$

where $\mathbf{s} \in \mathbb{Z}_q^n$ is the secret key, $\mathbf{a} \xleftarrow{\$} \mathbb{Z}_q^n$ is uniform, and e is a small Gaussian noise term with standard deviation σ. Zoo fixes $n = 630$, $q = 2^{32}$, and $\sigma = 2^{-15}$, yielding 128-bit security under standard LWE hardness assumptions [4].

2.2 Boolean Gate Evaluation

Homomorphic boolean gates (AND, OR, XOR, NOT) on LWE ciphertexts follow the gate-bootstrapping paradigm of Chillotti et al. [2]. For two input ciphertexts $\mathsf{ct}_0, \mathsf{ct}_1$:

$$\mathsf{ct}_{\text{AND}} = \mathsf{Bootstrap}((0, \lfloor q/8 \rfloor) - \mathsf{ct}_0 - \mathsf{ct}_1) \tag{2}$$

The bootstrap operation simultaneously evaluates a lookup table and refreshes noise, producing a ciphertext with bounded noise regardless of input noise level.

2.3 Programmable Bootstrapping

Zoo uses programmable bootstrapping (PBS) to evaluate arbitrary functions $f : \mathbb{Z}_p \to \mathbb{Z}_p$ during the bootstrap step itself. The test polynomial $v(X) \in \mathbb{Z}_q[X]/(X^N + 1)$ encodes f in its coefficients:

$$v(X) = \sum_{i=0}^{N-1} f\left(\left\lfloor \frac{i \cdot p}{2N} \right\rfloor\right) X^i \tag{3}$$

This eliminates post-bootstrap computation for common operations (sign extraction, ReLU, step functions), reducing circuit depth and noise accumulation.

2.4 Noise Management

Each homomorphic operation adds noise to the ciphertext. Zoo tracks noise budgets per ciphertext in the VM state:

Definition 1 (Noise Budget)**.** For ciphertext $\mathsf{ct} = (\mathbf{a}, b)$ with secret $\mathbf{s}$, the noise $e = b - \langle \mathbf{a}, \mathbf{s} \rangle - m \cdot \lfloor q/2 \rfloor$. The noise budget is $\beta = \lfloor \log_2(q/2) \rfloor - \lceil \log_2 |e| \rceil$ bits. Decryption succeeds when $\beta > 0$.

The VM rejects execution when a ciphertext's projected noise budget would drop below 1 bit, forcing the contract to insert a bootstrap before proceeding. Gas metering accounts for bootstrap cost.

3 CKKS on Zoo

3.1 Approximate Arithmetic

CKKS [3] encodes a vector $\mathbf{z} \in \mathbb{C}^{N/2}$ into a polynomial ring $R_q = \mathbb{Z}_q[X]/(X^N + 1)$ via the canonical embedding inverse, scaled by Δ:

$$m(X) = \lfloor \Delta \cdot \sigma^{-1}(\mathbf{z}) \rceil \in R_q \tag{4}$$

where $\Delta = 2^{40}$ is the scaling factor and σ^{-1} is the inverse canonical embedding. Encryption produces $\mathsf{ct} = (\mathsf{ct}_0, \mathsf{ct}_1) \in R_q^2$ under RLWE.

3.2 Homomorphic Operations

Addition of two CKKS ciphertexts at the same level is component-wise. Multiplication produces a degree-2 ciphertext that must be relinearized:

$$\mathsf{ct}_{\text{mult}} = (\mathsf{ct}_0^{(a)} \cdot \mathsf{ct}_0^{(b)},\ \mathsf{ct}_0^{(a)} \cdot \mathsf{ct}_1^{(b)} + \mathsf{ct}_1^{(a)} \cdot \mathsf{ct}_0^{(b)},\ \mathsf{ct}_1^{(a)} \cdot \mathsf{ct}_1^{(b)}) \tag{5}$$

$$\mathsf{ct}_{\text{relin}} = \mathsf{Relin}(\mathsf{ct}_{\text{mult}}, \mathsf{evk}) \tag{6}$$

3.3 Rescaling and Level Management

After multiplication, the scaling factor doubles to Δ^2. Rescaling divides by Δ and drops one prime from the ciphertext modulus chain:

$$\mathsf{Rescale}(\mathsf{ct}, q_\ell) \to \mathsf{ct}' \text{ at level } \ell - 1, \quad q_{\ell-1} = q_\ell / p_\ell \tag{7}$$

Zoo provisions a modulus chain of $L = 15$ levels by default, supporting up to 14 sequential multiplications—sufficient for a 4-layer neural network with ReLU approximations. The VM tracks levels alongside gas, rejecting operations that would exhaust the chain.

3.4 ML Inference on Encrypted Data

A neural-network forward pass on encrypted inputs uses CKKS slot packing. For a weight matrix $W \in \mathbb{R}^{m \times n}$ and encrypted input $\mathsf{Enc}(\mathbf{x})$:

$$\mathsf{Enc}(W\mathbf{x} + \mathbf{b}) = W \otimes \mathsf{Enc}(\mathbf{x}) \oplus \mathsf{Enc}(\mathbf{b}) \tag{8}$$

where $\otimes$ denotes plaintext–ciphertext multiplication and $\oplus$ denotes ciphertext addition. Activation functions are approximated by low-degree polynomials (degree-3 Chebyshev for ReLU) consuming one multiplicative level each.

4 GPU Acceleration

4.1 NTT Butterfly Parallelism

The computational bottleneck in both TFHE and CKKS is polynomial multiplication in R_q, implemented via the Number Theoretic Transform (NTT). The NTT butterfly network of depth $\log_2 N$ maps naturally to GPU thread blocks:

- Each butterfly stage is a parallel map over $N/2$ independent pairs.
- Stages synchronize via `__syncthreads()` within a block or global barrier across blocks.
- For $N = 2^{14} = 16384$ (Zoo's default polynomial degree), a single SM processes 8192 butterflies per stage.

4.2 TFHE Bootstrap on GPU

A single TFHE bootstrap consists of:

1. **Blind rotation**: $n = 630$ external products, each requiring two NTT forward/inverse transforms on degree-N polynomials.
2. **Key switching**: one matrix–vector product in $\mathbb{Z}_q^{n' \times n}$.
3. **Modulus switching**: rounding from q to q'.

On CPU (single-threaded, AVX-512), one bootstrap takes $\sim$12 ms. On an NVIDIA H100 GPU:

- Blind rotation: 630 external products execute as a batched NTT kernel, 0.22 ms.
- Key switching: cuBLAS GEMV, 0.05 ms.

- Modulus switching: element-wise kernel, 0.03 ms.
- **Total: 0.3 ms** (40× speedup).

4.3 Batch Amortization

When multiple independent bootstraps are required (e.g., evaluating a 256-bit comparison circuit), Zoo batches them into a single GPU dispatch:

$$T_{\text{batch}}(k) = T_{\text{single}} + \alpha \cdot k, \quad \alpha \approx 0.015\,\text{ms per additional bootstrap} \tag{9}$$

for $k \leq 4096$ bootstraps. Beyond this, L2 cache pressure causes throughput degradation. At $k = 256$ (a typical 8-bit integer comparison), total latency is ∼4.1 ms.

4.4 Memory Bounds

FHE ciphertexts are large. A single CKKS ciphertext at level $\ell = 15$ with $N = 16384$ occupies:

$$|\mathsf{ct}| = 2 \cdot N \cdot \ell \cdot 8 = 2 \cdot 16384 \cdot 15 \cdot 8 \approx 3.75\,\text{MB} \tag{10}$$

An H100 with 80 GB HBM3 can hold ∼21,000 such ciphertexts simultaneously. Zoo validators must provision at least 40 GB GPU memory for FHE workloads. The VM enforces per-contract ciphertext storage limits via gas metering.

5 Threshold Decryption

5.1 Motivation

In a blockchain setting, no single entity should hold the decryption key. Zoo distributes the TFHE/CKKS secret key across validators using Shamir secret sharing with a 67-of-100 reconstruction threshold.

5.2 Encrypt-to-Shares Protocol

Key generation proceeds as follows:

1. A trusted dealer (the genesis ceremony) generates $\mathbf{s} \in \mathbb{Z}_q^n$ and computes Shamir shares $\mathbf{s}_i$ for $i \in \{1, \ldots, 100\}$ with threshold $t = 67$.
2. Each share $\mathbf{s}_i$ is encapsulated under validator i's ML-KEM-768 public key and published to Lux K-Chain as a sealed transaction.
3. Validators decrypt their share locally and store it in a hardware security module (HSM) or TEE enclave.

Definition 2 (E2S Decryption)**.** To decrypt ciphertext $\mathsf{ct} = (\mathbf{a}, b)$, each participating validator i computes a partial decryption:

$$d_i = \langle \mathbf{a}, \mathbf{s}_i \rangle + e_i \tag{11}$$

where e_i is a smudging noise term with $\|e_i\| \ll q/4$ to prevent share leakage. The aggregator reconstructs:

$$m = b - \sum_{i \in S} \lambda_i \cdot d_i \pmod{q}, \quad |S| \geq 67 \tag{12}$$

where λ_i are Lagrange interpolation coefficients for the set S.

5.3 67-of-100 Default Threshold

The threshold $t = 67$ is chosen to satisfy:

- **Safety**: an adversary controlling fewer than 34 validators cannot decrypt any ciphertext.
- **Liveness**: the system tolerates up to 33 offline or Byzantine validators while maintaining decryption capability.
- **Alignment with BFT**: the $2/3 + 1$ threshold matches the Byzantine fault tolerance bound used in Zoo T-Chain consensus.

5.4 Key Share Rotation

Validator set changes trigger proactive secret sharing (PSS). New shares are computed via a re-sharing protocol without reconstructing $\mathbf{s}$:

$$\mathbf{s}_i^{(\text{new})} = \sum_{j \in S_{\text{old}}} \lambda_j \cdot \mathbf{s}_{j \to i} \tag{13}$$

where $\mathbf{s}_{j \to i}$ is a re-share from old validator j to new validator i, again encapsulated under ML-KEM-768. This ensures forward secrecy: compromised old shares cannot decrypt ciphertexts encrypted under the new epoch's public key.

6 Integration with T-Chain

Zoo T-Chain (Threshold Chain) combines threshold signing [13] with FHE in a unified VM execution environment.

6.1 FHE Opcodes

The T-Chain VM extends the EVM with FHE-specific opcodes:

Opcode	Operation	Gas Cost
`FHE_ENC`	Encrypt plaintext to TFHE/CKKS ciphertext	50,000
`FHE_ADD`	Homomorphic addition	10,000
`FHE_MUL`	Homomorphic multiplication (+ relin)	200,000
`FHE_BOOT`	Programmable bootstrap (TFHE)	500,000
`FHE_RESCALE`	Rescale CKKS ciphertext	80,000
`FHE_ROTATE`	Rotate CKKS slots	150,000
`FHE_DEC_REQ`	Request threshold decryption	1,000,000

Table 1: FHE opcodes added to the T-Chain VM

Gas costs reflect computational load. `FHE_DEC_REQ` is the most expensive because it triggers a multi-round threshold decryption protocol across 67+ validators.

6.2 GPU Acceleration Configuration

Validators configure GPU FHE acceleration in their node configuration:

- `fhe.gpu.enabled`: boolean, default `true` for T-Chain validators.

- `fhe.gpu.device`: CUDA device index (supports multi-GPU).
- `fhe.gpu.max_batch`: maximum bootstrap batch size (default 4096).
- `fhe.gpu.memory_limit`: GPU memory reserved for FHE (default 40 GB).

Validators without GPU support may still participate in consensus and threshold decryption but cannot execute FHE opcodes. Block producers for FHE-heavy blocks are selected preferentially from the GPU-enabled validator subset.

6.3 Ciphertext Storage

TFHE ciphertexts (LWE, ∼5 KB each) and CKKS ciphertexts (∼3.75 MB at full level) are stored in a dedicated ciphertext trie separate from the account state trie. Content-addressing (Blake3 hash) enables deduplication. State rent is charged per ciphertext per epoch to bound storage growth.

7 Formal Verification

Zoo maintains Lean 4 proofs for critical FHE subsystems, following the broader Zoo formal verification initiative [12].

7.1 CKKS Correctness

The file `Crypto/FHE/CKKS.lean` proves:

Theorem 1 (Rescaling Precision)**.** *For a* CKKS *ciphertext at level ℓ with scaling factor Δ and modulus q_ℓ, rescaling to level $\ell - 1$ introduces approximation error bounded by:*

$$\|\epsilon_{\text{rescale}}\|_\infty \leq \frac{N \cdot B_{\text{key}}}{p_\ell} \tag{14}$$

where B_{key} bounds the secret key coefficients and p_ℓ is the dropped prime.

7.2 TFHE Noise Tracking

The file `Crypto/FHE/TFHE.lean` verifies:

Theorem 2 (Bootstrap Noise Bound)**.** *After programmable bootstrapping with parameters $(n, N, q, \sigma, B_g, d_g)$, the output noise satisfies:*

$$\|e_{\text{out}}\| \leq n \cdot d_g \cdot (N+1) \cdot B_g \cdot \sigma_{\text{bsk}} + (1 + N \cdot \|\mathbf{s}\|_1) \cdot \frac{B_g^{d_g}}{2} \tag{15}$$

This bound is strictly less than $q/4$ for Zoo*'s parameter choices, guaranteeing correct decryption.*

7.3 GPU Scaling

The file `GPU/FHEScaling.lean` proves:

Proposition 3 (NTT Parallelism)**.** *The $\log_2 N$-stage butterfly network for degree-N NTT admits a work-depth decomposition with:*

$$W = \frac{N}{2} \log_2 N, \quad D = \log_2 N \tag{16}$$

Given $P \geq N/2$ parallel processors, the time complexity is $\Theta(\log_2 N)$, achieving linear speedup up to $P = N/2$.

8 Applications

8.1 Confidential DeFi

Encrypted order matching. Users submit encrypted limit orders $\mathsf{Enc}(\mathit{price}, \mathit{quantity})$. The matching engine evaluates comparison circuits homomorphically (TFHE for price comparison, CKKS for quantity arithmetic). Matched orders are decrypted via threshold decryption; unmatched orders remain encrypted and invisible to other participants.

This eliminates front-running, sandwich attacks, and information leakage from the order book.

8.2 Private AI Inference

A model owner publishes encrypted weights $\mathsf{Enc}(W_i)$ and a user submits encrypted input $\mathsf{Enc}(\mathbf{x})$. The contract evaluates the forward pass entirely under CKKS encryption:

$$\mathsf{Enc}(\hat{y}) = f_{\mathrm{NN}}(\mathsf{Enc}(W_1), \ldots, \mathsf{Enc}(W_L), \mathsf{Enc}(\mathbf{x})) \tag{17}$$

Neither the model weights nor the input are revealed to any party. The user decrypts $\hat{y}$ locally. This enables monetization of proprietary models without exposing intellectual property.

8.3 Sealed-Bid Auctions

Bidders submit $\mathsf{Enc}(\mathit{bid}_i)$ to an auction contract. The contract computes the maximum via a homomorphic comparison tree (TFHE with $\lceil \log_2 k \rceil$ rounds for k bids). Only the winning bid is decrypted; losing bids remain sealed.

9 Benchmarks

All benchmarks run on Zoo testnet validators equipped with NVIDIA H100 80 GB GPUs, AMD EPYC 9654 CPUs (96 cores), and 512 GB DDR5.

9.1 Bootstrap Latency

Batch amortization yields superlinear speedup because GPU occupancy increases with batch size until the 4096-element sweet spot.

Operation	CPU (ms)	GPU (ms)	Speedup
Single TFHE bootstrap	12.0	0.30	40×
8-bit integer comparison	96.0	4.10	23×
256-bit comparison circuit	3,072	52.0	59×
Batch 4096 bootstraps	49,152	62.0	793×

Table 2: TFHE bootstrap latency: CPU vs GPU

Metric	CPU	GPU
Single ciphertext multiply + relin (ms)	8.5	0.6
Throughput (multiplications/sec)	118	1,667
Rescale (ms)	2.1	0.15
Slot rotation (ms)	6.2	0.45

Table 3: CKKS operation throughput at $N = 16384$, $L = 15$

9.2 CKKS Multiplication Throughput

9.3 Threshold Decryption Latency

The dominant cost is network latency for collecting 67 partial decryptions. Computation is negligible relative to communication.

9.4 End-to-End Application Benchmarks

10 Related Work

Lux TFHE. The Lux `lux-tfhe` paper [9] introduced GPU-accelerated TFHE for the Lux L0 base layer. Zoo extends this work with threshold decryption, CKKS support, and deeper VM integration via FHE-specific opcodes on T-Chain.

Zama fhEVM. Zama's fhEVM [6] pioneered FHE smart contracts on EVM-compatible chains. Zoo differs in three respects: (1) CKKS support for approximate arithmetic, enabling ML inference; (2) threshold decryption rather than single-operator key management; (3) formal verification of noise bounds and GPU scaling in Lean 4.

Sunscreen FHE. The Sunscreen compiler [8] provides a Rust-based FHE toolkit targeting BFV and CKKS. Zoo builds on similar compiler infrastructure but integrates it directly into the blockchain VM with gas metering and threshold key management.

Concrete/TFHE-rs. Zama's `tfhe-rs` library [7] provides a Rust implementation of TFHE with GPU support. Zoo's GPU kernels build on this foundation, extending it with batched bootstrapping optimizations and T-Chain integration.

Threshold FHE in MPC. Boneh et al. [5] formalized threshold FHE for multi-party computation. Zoo adapts this framework to the blockchain setting, replacing interactive MPC rounds with asynchronous partial-decryption collection anchored by BFT consensus.

Phase	Latency (ms)
Partial decryption (per validator)	0.8
Network collection (67 shares, p95)	180
Lagrange aggregation	2.1
Smudging noise verification	0.3
Total end-to-end	**∼183**

Table 4: Threshold decryption latency for LWE ciphertext (67-of-100)

Application	Latency (ms)	Gas Cost
Encrypted order match (2 orders)	48	1.8M
Sealed-bid auction (64 bids)	312	42M
4-layer NN inference (CKKS)	580	68M

Table 5: End-to-end application benchmarks on GPU-enabled validators

11 Conclusion

We have presented Zoo Network's threshold FHE system, combining TFHE for boolean circuits and CKKS for approximate arithmetic with GPU acceleration and validator-distributed threshold decryption. Key results:

1. **40× GPU speedup** for single TFHE bootstrap (12 ms → 0.3 ms), with up to 793× for batched operations.
2. **67-of-100 threshold decryption** aligned with BFT consensus, with end-to-end latency of ∼183 ms dominated by network communication.
3. **Formal verification** of CKKS rescaling precision, TFHE noise bounds, and GPU parallelism in Lean 4.
4. **Practical applications** demonstrated: encrypted order matching (48 ms), sealed-bid auctions (312 ms), and private neural-network inference (580 ms).

FHE on Zoo T-Chain enables a new class of privacy-preserving smart contracts where computation proceeds on encrypted state without any party—including validators—observing plaintext values. Combined with post-quantum key encapsulation (ML-KEM-768) for share distribution, the system provides long-term confidentiality against both classical and quantum adversaries.

Future work includes bootstrapping-free FHE schemes for reduced latency, hardware-accelerated key switching via custom FPGA pipelines, and integration with Zoo's experience ledger [10] for privacy-preserving model training.

Acknowledgments

The authors thank the Zoo Labs Foundation engineering team for GPU kernel development, the Lux cryptography team for the `lux-tfhe` foundation, and the Hanzo infrastructure team for validator provisioning.

References

[1] Craig Gentry. *Fully Homomorphic Encryption Using Ideal Lattices*. STOC, 2009.

[2] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. *TFHE: Fast Fully Homomorphic Encryption over the Torus*. Journal of Cryptology, 33(1):34–91, 2020.

[3] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. *Homomorphic Encryption for Arithmetic of Approximate Numbers*. ASIACRYPT, 2017.

[4] Martin R. Albrecht, Rachel Player, and Sam Scott. *On the Concrete Hardness of Learning with Errors*. Journal of Mathematical Cryptology, 9(3):169–203, 2015.

[5] Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M.R. Rasmussen, and Amit Sahai. *Threshold Cryptosystems from Threshold Fully Homomorphic Encryption*. CRYPTO, 2018.

[6] Zama. *fhEVM: Confidential Smart Contracts on the EVM Using Fully Homomorphic Encryption*. Zama Whitepaper, 2023.

[7] Zama. *TFHE-rs: A Pure Rust Implementation of the TFHE Scheme*. `https://github.com/zama-ai/tfhe-rs`, 2023.

[8] Sunscreen. *Sunscreen: A Compiler for FHE Programs*. Sunscreen Tech Report, 2023.

[9] Lux Industries. *GPU-Accelerated TFHE for Lux Network*. Lux Research, 2025.

[10] Zach Kelling. *Zoo Network Tokenomics: Fair Distribution Through Complete Airdrop*. Zoo Labs Foundation, 2025.

[11] Zach Kelling. *Zoo EVM: A Post-Quantum Safe Layer 2 Blockchain with AI-Native Precompiles*. Zoo Labs Foundation, 2026.

[12] Zach Kelling. *Zoo Network: Decentralized AI Training and Inference Layer with Semantic Optimization*. Zoo Labs Foundation, 2025.

[13] Zach Kelling. *Threshold Signatures on Zoo T-Chain* (forthcoming). Zoo Labs Foundation, 2026.